

CS217: Artificial Intelligence and Machine Learning (associated lab: CS240)

Pushpak Bhattacharyya
CSE Dept.,
IIT Bombay

Week5 of 3feb25, sigmoid, softmax

Main points covered: week3 of
27jan25

Fundamental Observation

- The number of TFs computable by a perceptron is equal to the number of regions produced by 2^n hyper-planes, obtained by plugging in the values $\langle x_1, x_2, x_3, \dots, x_n \rangle$ in the equation

$$\sum_{i=1}^n w_i x_i = \theta$$

Number of regions founded by n hyperplanes in d-dim passing through origin is given by the following recurrence relation

$$R_{n,d} = R_{n-1,d} + R_{n-1,d-1}$$

we use generating function as an operating function

Boundary condition:

$$\begin{array}{ll} R_{1,d} = 2 & \text{1 hyperplane in d-dim} \\ R_{n,1} = 2 & \text{n hyperplanes in 1-dim,} \\ & \text{Reduce to n points thru origin} \end{array}$$

The generating function is

$$f(x,y) = \sum_{n=1}^{\infty} \sum_{d=1}^{\infty} R_{n,d} \cdot x^n y^d$$

Comparing co-efficients we get

$$R_n, d = 2 \sum_{i=0}^{d-1} C_i^{n-1}$$

Implication

- $R(n,d)$ becomes for a perceptron with m weights and 1 threshold $R(2^m, m+1)$

$$= 2 \sum_{i=0}^{m+1-1} C_i^{2^m-1}$$

$$= 2 \sum_{i=0}^m C_i^{2^m-1}$$

$$= O(2^{m^2})$$

- Total no of Boolean Function is 2^{2^m} . Shows why $\#TF \ll \#BF$

Gradient Descent Technique

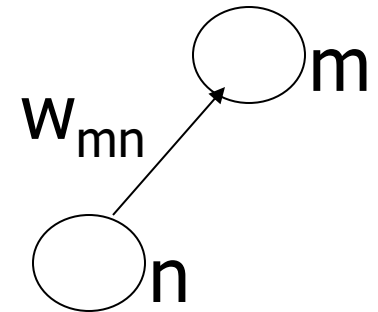
- Let E be the error at the output layer

$$E = \frac{1}{2} \sum_{j=1}^p \sum_{i=1}^n (t_i - o_i)_j^2$$

- t_i = target output; o_i = observed output
- i is the index going over n neurons in the outermost layer
- j is the index going over the p patterns (1 to p)
- Ex: XOR:— $p=4$ and $n=1$

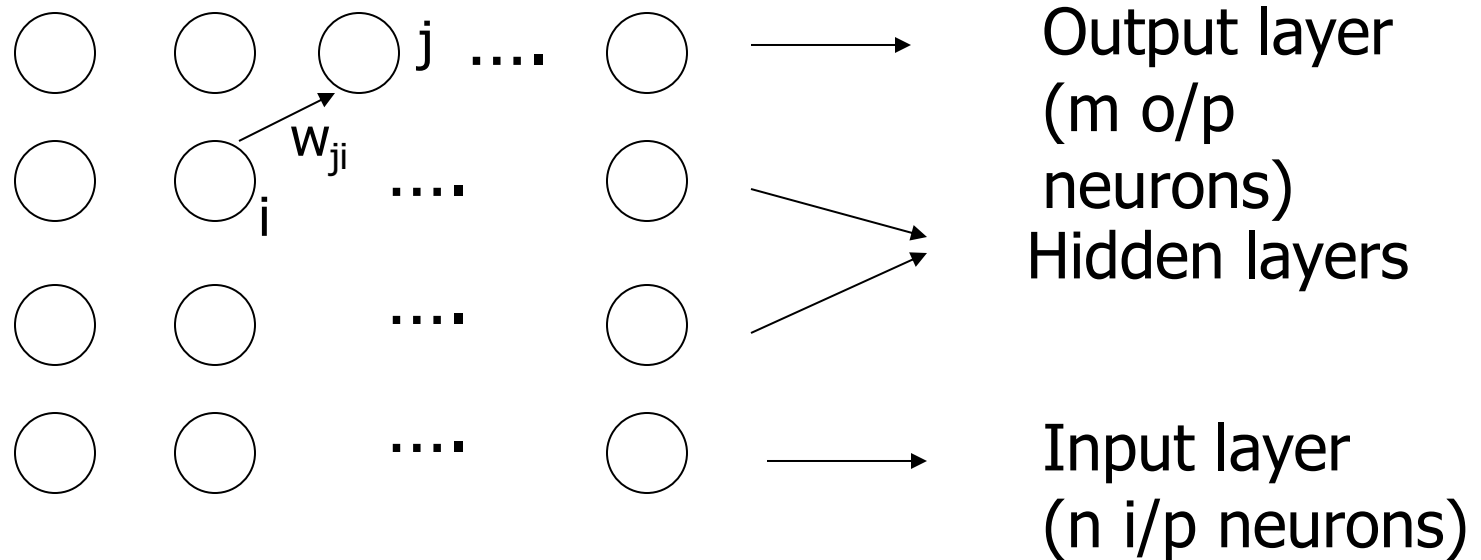
Weights in a FF NN

- w_{mn} is the weight of the connection from the n^{th} neuron to the m^{th} neuron
- E vs $\bar{w}$ surface is a complex surface in the space defined by the weights w_{ij}
- $-\frac{\delta E}{\delta w_{mn}}$ gives the direction in which a movement of the operating point in the w_{mn} co-ordinate space will result in maximum decrease in error



$$\Delta w_{mn} \propto -\frac{\delta E}{\delta w_{mn}}$$

Backpropagation algorithm



- Fully connected feed forward network
- Pure FF network (no jumping of connections over layers)

General Backpropagation Rule

- General weight updating rule:

$$\Delta w_{ji} = \eta \delta_j o_i$$

- Where

$$\delta_j = (t_j - o_j) o_j (1 - o_j) \quad \text{for outermost layer}$$

$$= \sum_{k \in \text{nextlayer}} (w_{kj} \delta_k) o_j (1 - o_j) o_i \quad \text{for hidden layers}$$

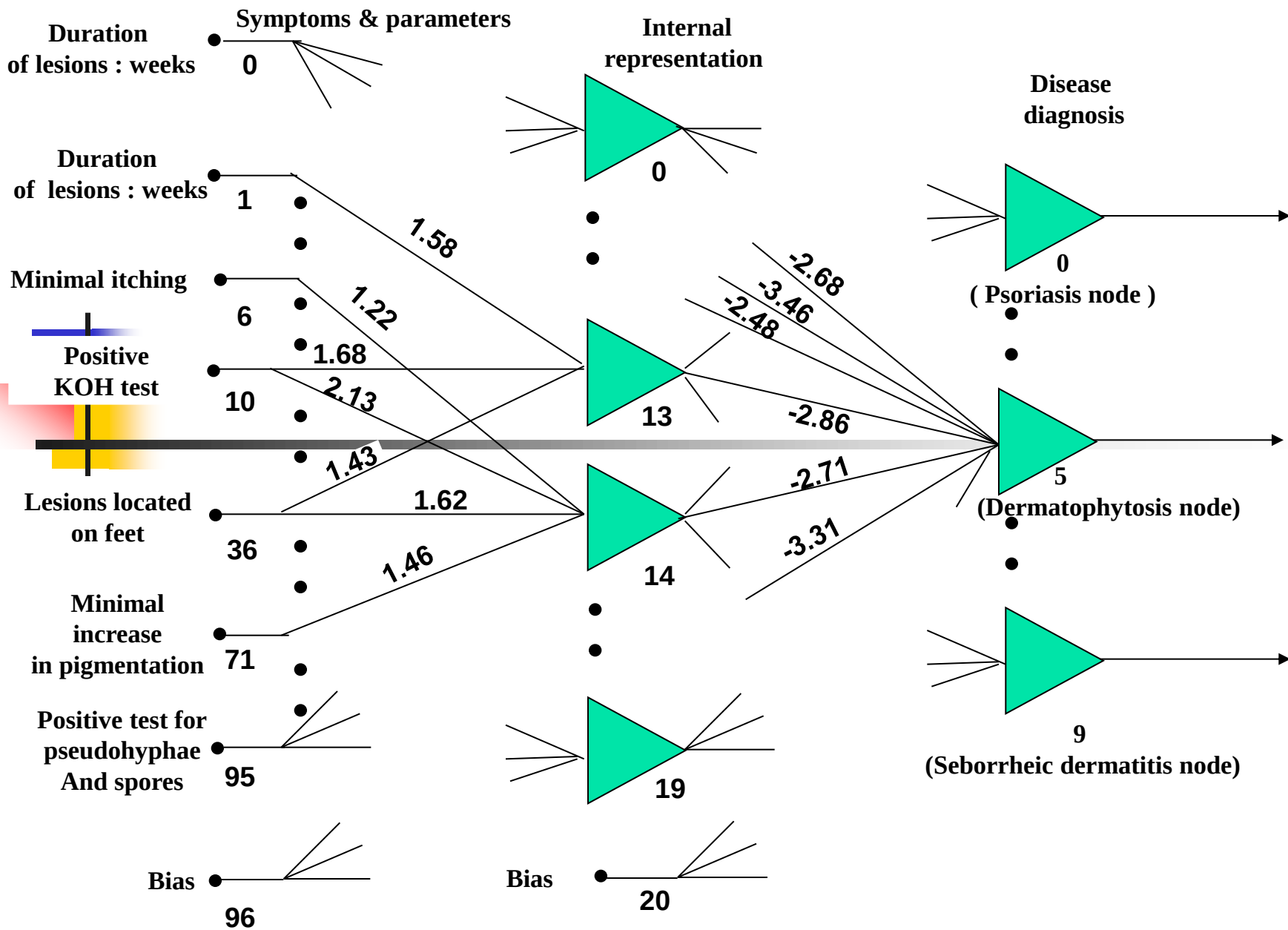
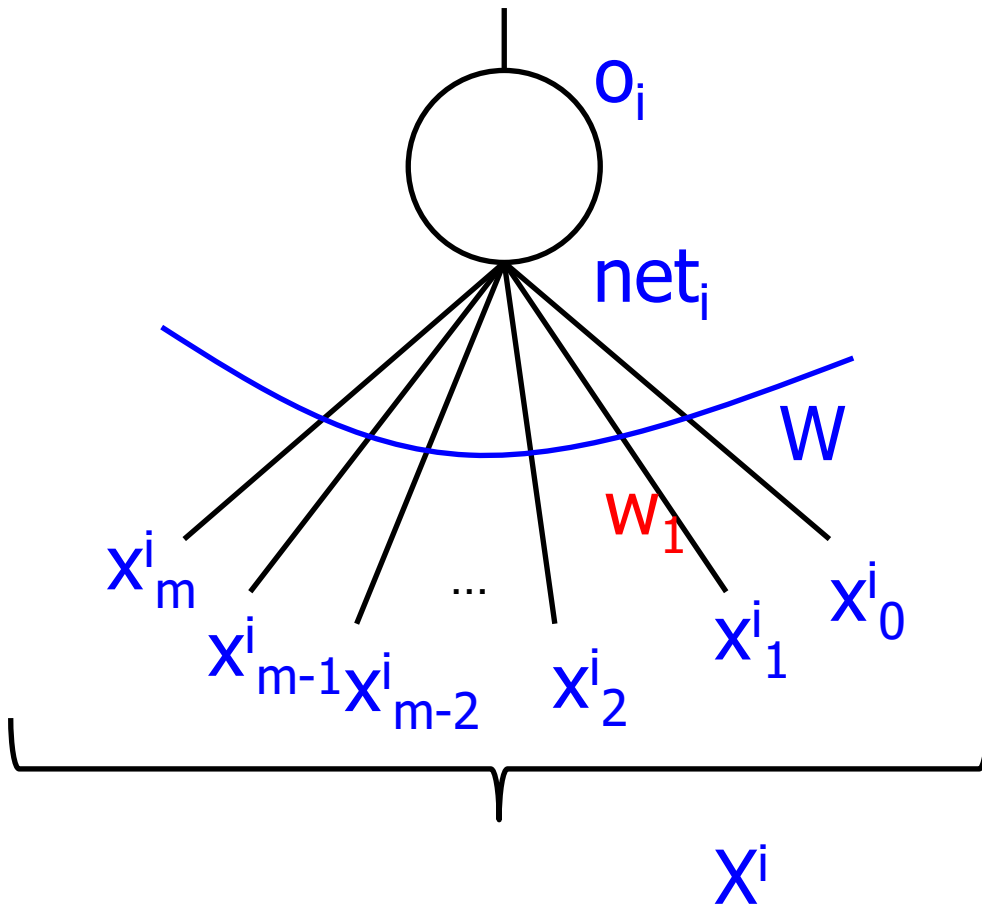


Figure : Explanation of dermatophytosis diagnosis using the DESKNET expert system.

End of main points

Sigmoid

Sigmoid neuron



$$o^i = \frac{1}{1 + e^{-net^i}}$$

$$net_i = W \cdot X^i = \sum_{j=0}^m w_j x_j^i$$

Sigmoid function: can saturate

- Brain saving itself from itself, in case of extreme agitation, emotion etc.



Definition: Sigmoid or Logit function

$$y = \frac{1}{1 + e^{-x}}$$

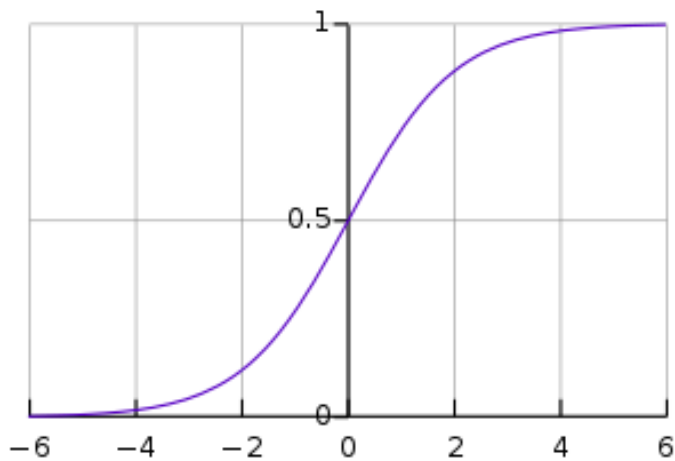
$$\frac{dy}{dx} = y(1 - y)$$

$$y = \frac{1}{1 + e^{-kx}}$$

$$\frac{dy}{dx} = ky(1 - y)$$

If k tends to infinity, sigmoid tends to the step function

Sigmoid function



$$f(x) = \frac{1}{1+e^{-x}}$$

$$\begin{aligned} f(x) &= \frac{1}{1+e^{-x}} \\ \frac{df(x)}{dx} &= \frac{d}{dx} \left(\frac{1}{1+e^{-x}} \right) \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}} \right) \\ &= f(x) \cdot (1 - f(x)) \end{aligned}$$

Decision making under sigmoid

- Output of sigmoid is between 0-1
- Look upon this value as probability of Class-1 (C_1)
- $1-\text{sigmoid}(x)$ is the probability of Class-2 (C_2)
- Decide C_1 , if $P(C_1) > P(C_2)$, else C_2

Sigmoid function and multiclass classification

- Why can't we use sigmoid for n-class classification? Have segments on the curve devoted to different classes, just like $-\infty$ to 0.5 is for class 2 and 0.5 to plus infinity is class 2.
- Think about it!!

multiclass: SOFTMAX

- 2-class $\rightarrow$ multi-class (C classes)
- Sigmoid $\rightarrow$ softmax
- i^{th} input, c^{th} class (small c), c varies over classes
- In softmax, decide for that class which has the highest probability

What is softmax

- Turns a vector of K real values into a vector of K real values that sum to 1
- Input values can be positive, negative, zero, or greater than one
- But softmax transforms them into values between 0 and 1
- so that they can be interpreted as probabilities.

Mathematical form

$$\sigma(\vec{Z})_i = \frac{e^{Z_i}}{\sum_{j=1}^K e^{Z_j}}$$

- σ is the **softmax** function
- Z is the input vector of size K
- The RHS gives the i^{th} component of the output vector
- Input to softmax and output of softmax are of the same dimension

Example

$$\bar{Z} = \langle 1, 2, 3 \rangle$$

$$Z_1 = 1, Z_2 = 2, Z_3 = 3$$

$$e^1 = 2.72, e^2 = 7.39, e^3 = 20.09$$

$$\sigma(\bar{Z}) = \left\langle \frac{2.72}{2.72 + 7.39 + 20.09}, \frac{7.39}{2.72 + 7.39 + 20.09}, \frac{20.09}{2.72 + 7.39 + 20.09} \right\rangle$$
$$= \langle .09, 0.24, 0.67 \rangle$$

Softmax and Cross Entropy

- Intimate connection between softmax and cross entropy
- Softmax gives a vector of probabilities
- Winner-take-all strategy will give a classification decision

Winner-take-all with softmax

- Consider the softmax vector obtained from the example where the softmax vector is $\langle 0.09, 0.24, 0.65 \rangle$
- These values correspond to 3 classes
 - For example, - *positive (+), negative (-)* and *neutral (0)* sentiments, given an input sentence like
 - *(a) I like the story line of the movie (+). (b) However the acting is weak (-). (c) The protagonist is a sports coach (0)*

Sentence vs. Sentiment

Sentence vs. Sentiment	Positive <i>(a) I like the story line of the movie (+). (b) However the acting is weak (-). (c) The protagonist is a sports coach (0)</i>	Negative	Neutral
Sent (a)	1 <i>(P_{max} from softmax)</i>	0	0
Sentence (b)	0	1 <i>(P_{max} from softmax)</i>	0
Sentence (C)	0	0`	1 <i>(Pmax from softmax)</i>

Training data

- *(a) I like the story line of the movie (+).*
- *(b) However the acting is weak (-).*
- *(c) The protagonist is a sports coach (0)*

Input

(a)

(b)

(c)

Output

$\langle 1, 0, 0 \rangle$

$\langle 0, 1, 0 \rangle$

$\langle 0, 0, 1 \rangle$

Finding the error

- Difference between target (T) and obtained (Y)
- Difference is called **LOSS**
- Options:
 - Total Sum Square Loss (TSS)
 - Cross Entropy (*measures difference between two probability distributions*)
- Softmax goes with cross entropy

Cross Entropy Function

$$H(P, Q) = - \sum_{x=1, N} \sum_{k=1, C} P(x, k) \log_2 Q(x, k)$$

x varies over N data instances, c varies over C classes
 P is target distribution; Q is observed distribution

Cross Entropy Loss

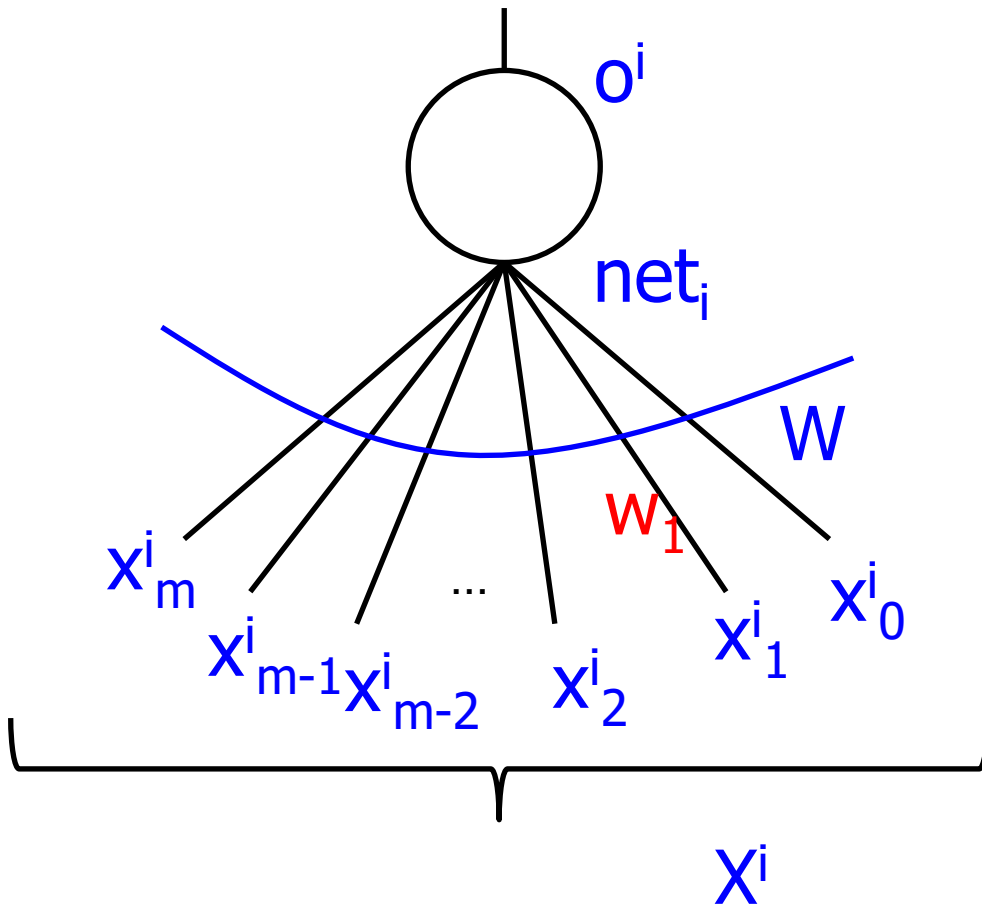
- Can we sum up cross entropies over the instances?
Is it allowed?
- Yes, summing up cross entropies (i.e. the total cross entropy loss) is equivalent to multiplying probabilities.
- Minimizing the total cross entropy loss is equivalent to maximizing the likelihood of observed data.

How to minimize loss

- Gradient descent approach
- Backpropagation Algorithm
- Involves derivative of the input-output function for each neuron
- FFNN with BP is the most important **TECHNIQUE** for us in the course

Sigmoid and Softmax neurons

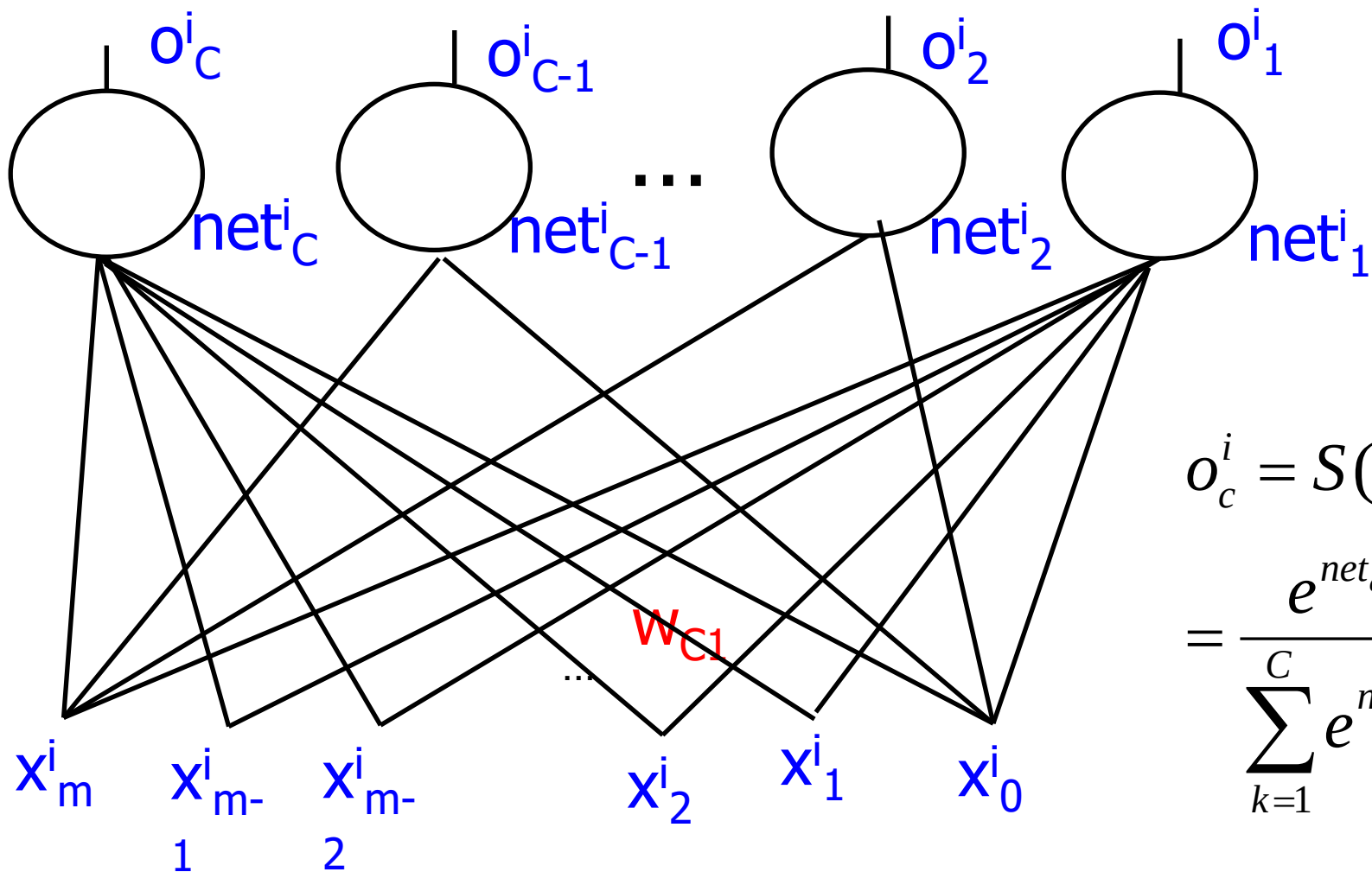
Sigmoid neuron



$$o^i = \frac{1}{1 + e^{-net^i}}$$

$$net_i = W \cdot X^i = \sum_{j=0}^m w_j x_j^i$$

Softmax Neuron



$$o_c^i = S(NET^i)_c$$

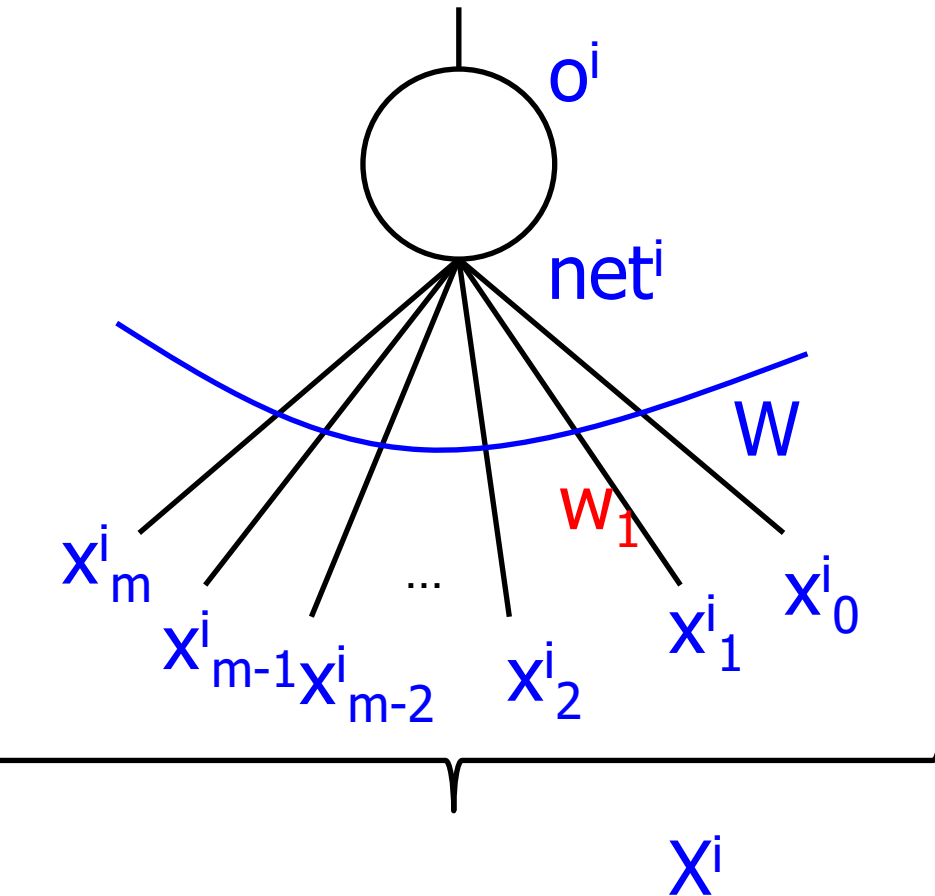
$$= \frac{e^{net_c^i}}{\sum_{k=1}^C e^{net_k^i}}$$

Output for class c (small c), c:1 to C

Notation

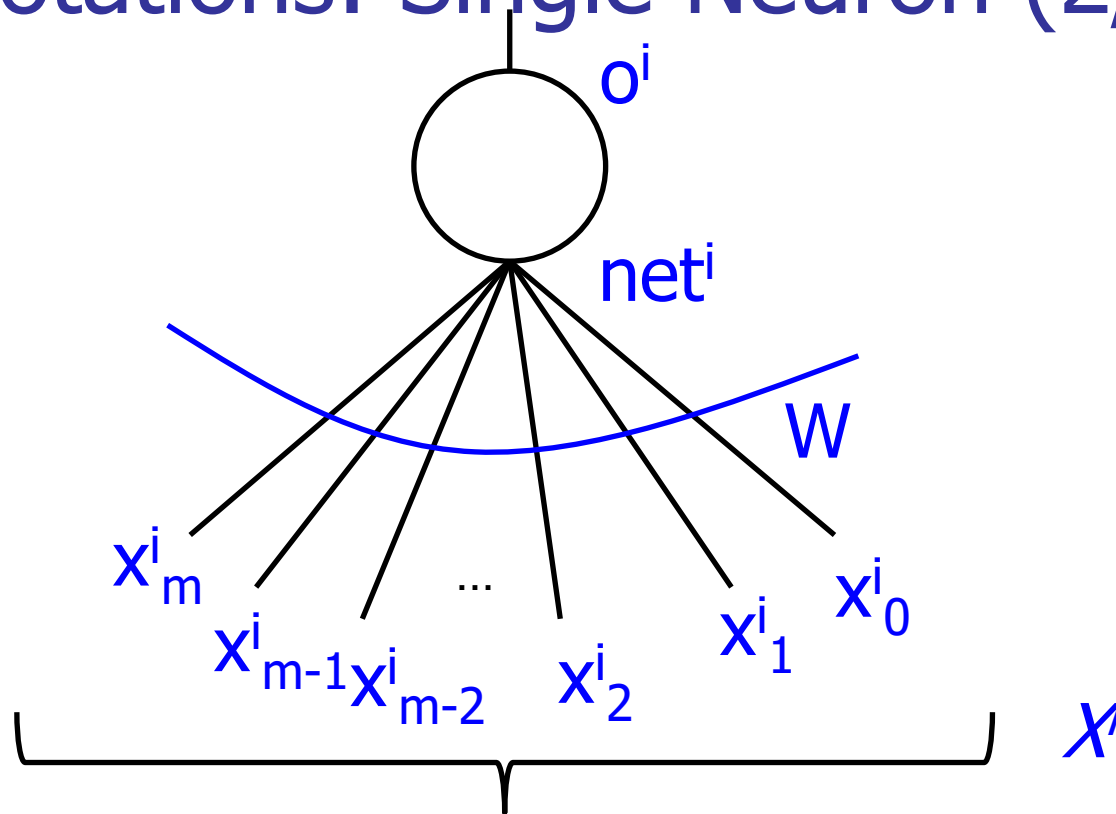
- $i=1..N$
- N i-o pairs, i runs over the training data
- $j=0...m$, m components in the input vector, j runs over the input dimension (also weight vector dimension)
- $k=1...C$, C classes (C components in the output vector)

Fix Notations: Single Neuron (1/2)



- Capital letter for vectors
- Small letter for scalars (therefore for vector components)
- X^i : i^{th} input vector
- o_i : output (scalar)
- W : weight vector
- net_i : $W \cdot X^i$
- There are n input-output observations

Fix Notations: Single Neuron (2/2)



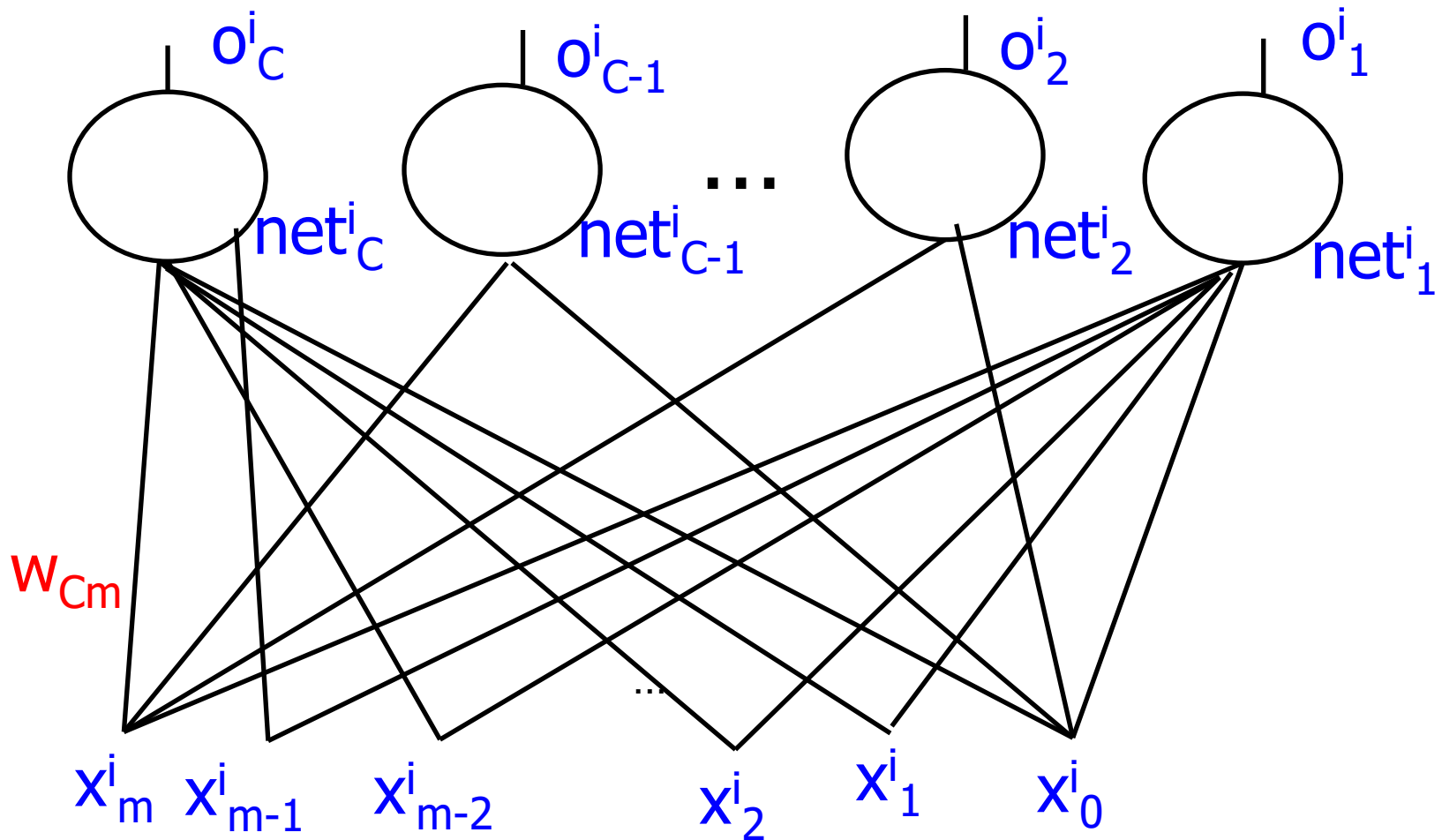
W and each X^i has m components

$W: \langle W_m, W_{m-1}, \dots, W_2, W_0 \rangle$

$X^i: \langle x^i_m, x^i_{m-1}, \dots, x^i_2, x^i_0 \rangle$

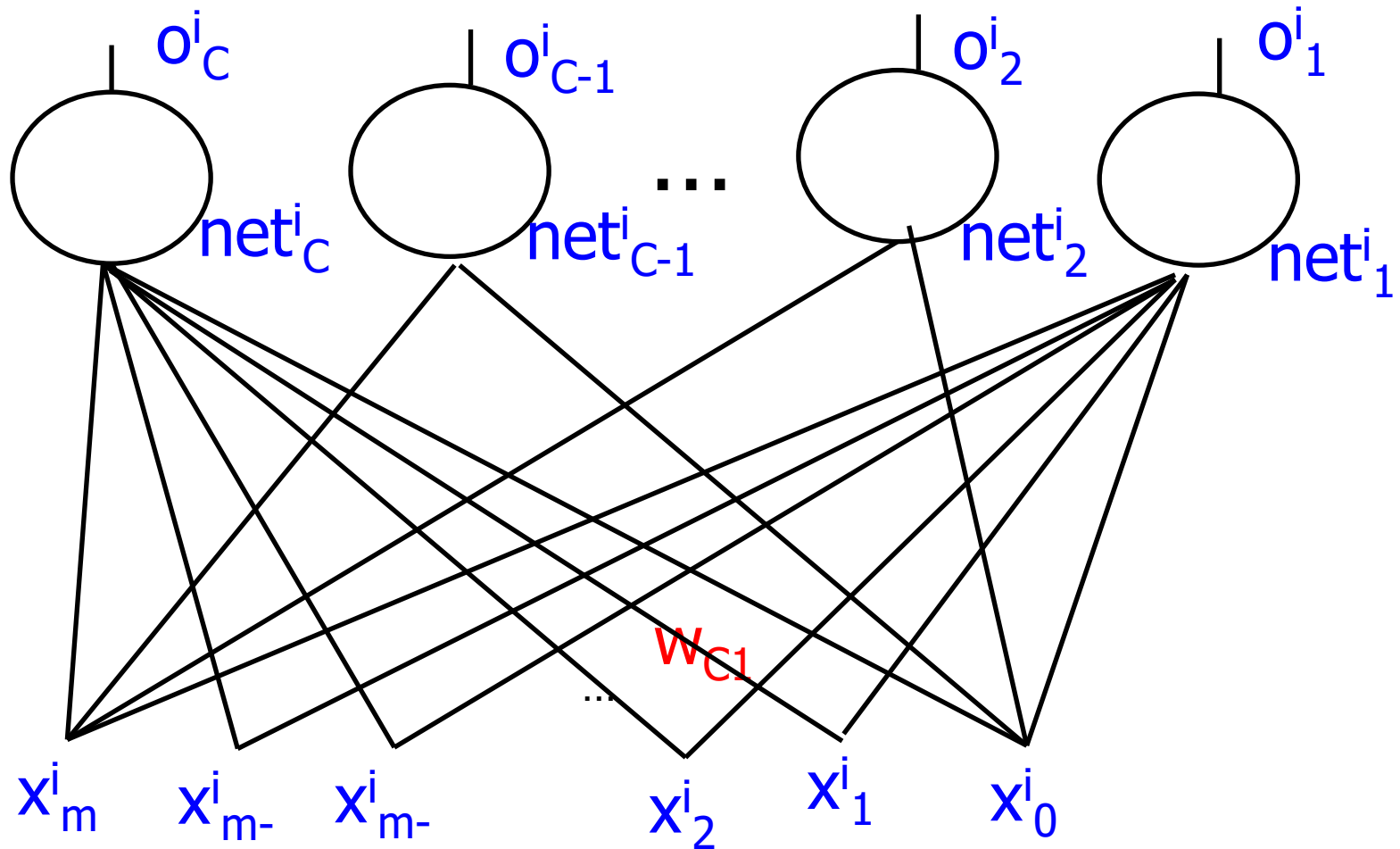
Upper suffix i indicates i^{th} input

Fixing Notations: Multiple neurons in o/p layer



Now, O^i and NET^i are vectors for i^{th} input
 W_c is the weight vector for c^{th} output neuron, $c=1..C$

Fixing Notations



Target Vector, T^i : $\langle t_c^i t_{c-1}^i \dots t_2^i t_1^i \rangle$, $i \rightarrow$ for i^{th} input.
 Only one of these C componets is 1, rest are 0

Derivatives

Derivative of sigmoid

$$o^i = \frac{1}{1 + e^{-net^i}}, \text{ for } i^{th} \text{ input}$$

$$\ln o^i = -\ln(1 + e^{-net^i})$$

$$\frac{1}{o^i} \frac{\partial o^i}{\partial net^i} = -\frac{1}{1 + e^{-net^i}} \cdot -e^{-net^i} = \frac{e^{-net^i}}{1 + e^{-net^i}} = (1 - o^i)$$

$$\Rightarrow \frac{\partial o^i}{\partial net^i} = o^i (1 - o^i)$$

Derivative of Softmax

$$o_c^i = \frac{e^{net_c^i}}{\sum_{k=1}^C e^{net_k^i}}, \text{ } i^{th} \text{ input pattern}$$

Derivative of Softmax: Case-1, class c for O and NET same

$$\ln o_c^i = net_c^i - \ln\left(\sum_{k=1}^C e^{net_k^i}\right)$$

$$\frac{1}{o_c^i} \frac{\partial o_c^i}{\partial net_c^i} = 1 - \frac{1}{\sum_{k=1}^C e^{net_k^i}} \cdot e^{net_c^i} = 1 - o_c^i$$

$$\Rightarrow \frac{\partial o_c^i}{\partial net_c^i} = o_c^i (1 - o_c^i)$$

Derivative of Softmax: Case-2,
class c' in net_c^i different from
class c of O

$$\ln o_c^i = net_c^i - \ln\left(\sum_{k=1}^C e^{net_k^i}\right)$$

$$\frac{1}{o_c^i} \frac{\partial o_c^i}{\partial net_{c'}^i} = 0 - \frac{1}{\sum_{k=1}^C e^{net_k^i}} \cdot e^{net_{c'}^i} = -o_{c'}^i$$

$$\Rightarrow \frac{\partial O_c^i}{\partial net_{c'}^i} = -o_c^i o_{c'}^i$$

Finding weight change rule

Foundation: Gradient descent

Change in weight $\Delta w_{ji} = -\eta \delta L / \delta w_{ji}$

$\eta =$ learning rate,
 $L =$ loss, $w_{ji} =$ weight of
connection from the i^{th}
neuron to j^{th}

At A, $\delta L / \delta w_{ji}$ is negative, so
 Δw_{ji} is positive.

At B, $\delta L / \delta w_{ji}$ is positive,
so Δw_{ji} is negative.

*E always decreases.
Greedy algo.*



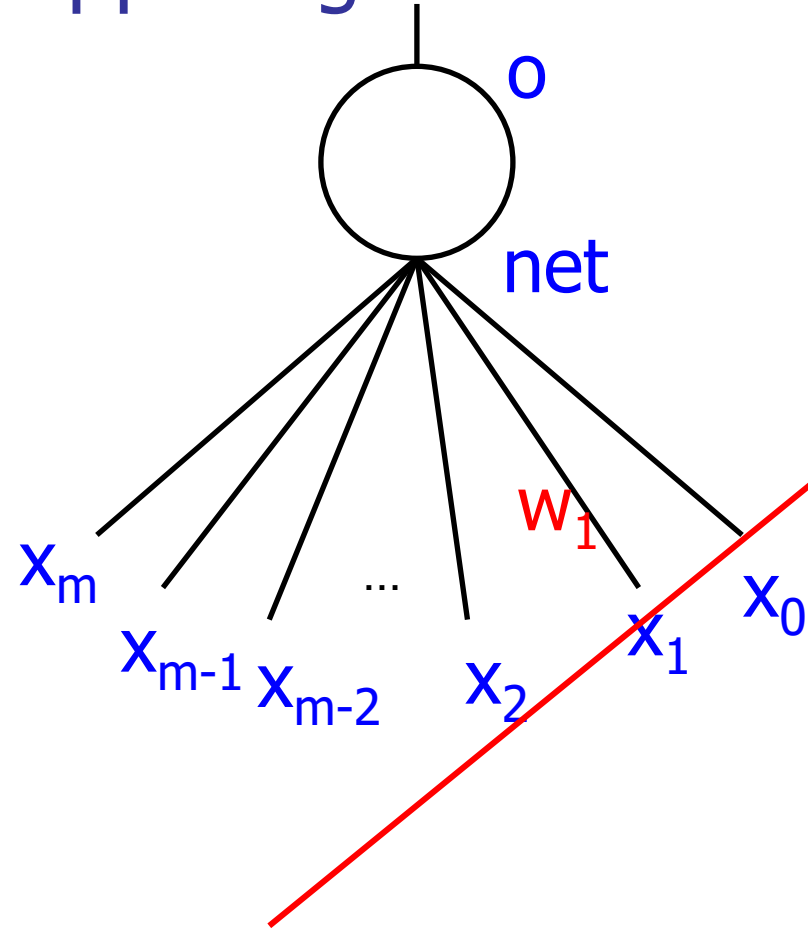
Gradient Descent is Greedy!

- Gradient Descent is greedy- always moves in the direction of reducing error
- Probabilistically also move in the direction of increasing error, to be able to come out of local minimum
- Nature randomly introduces some variation, and a totally new species emerges
- Darwin's theory of evolution

Genetic Algorithm

- Genetic Algorithms: adaptive heuristic search algorithms
- used to generate high-quality solutions for optimization problems and search problems
- To evolve the generation, genetic algorithms use the following operators, all PROBABILISTICALLY
 - Selection, Cross over, Mutation

Single sigmoid neuron and *cross entropy* loss, derived for single data point, hence dropping upper right suffix i



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial net} \cdot \frac{\partial net}{\partial w_1}$$

$$L = -t \log o - (1-t) \log(1-o)$$

$$\Rightarrow \frac{\partial L}{\partial o} = -\frac{t}{o} + \frac{1-t}{1-o} = -\frac{t-o}{o(1-o)}$$

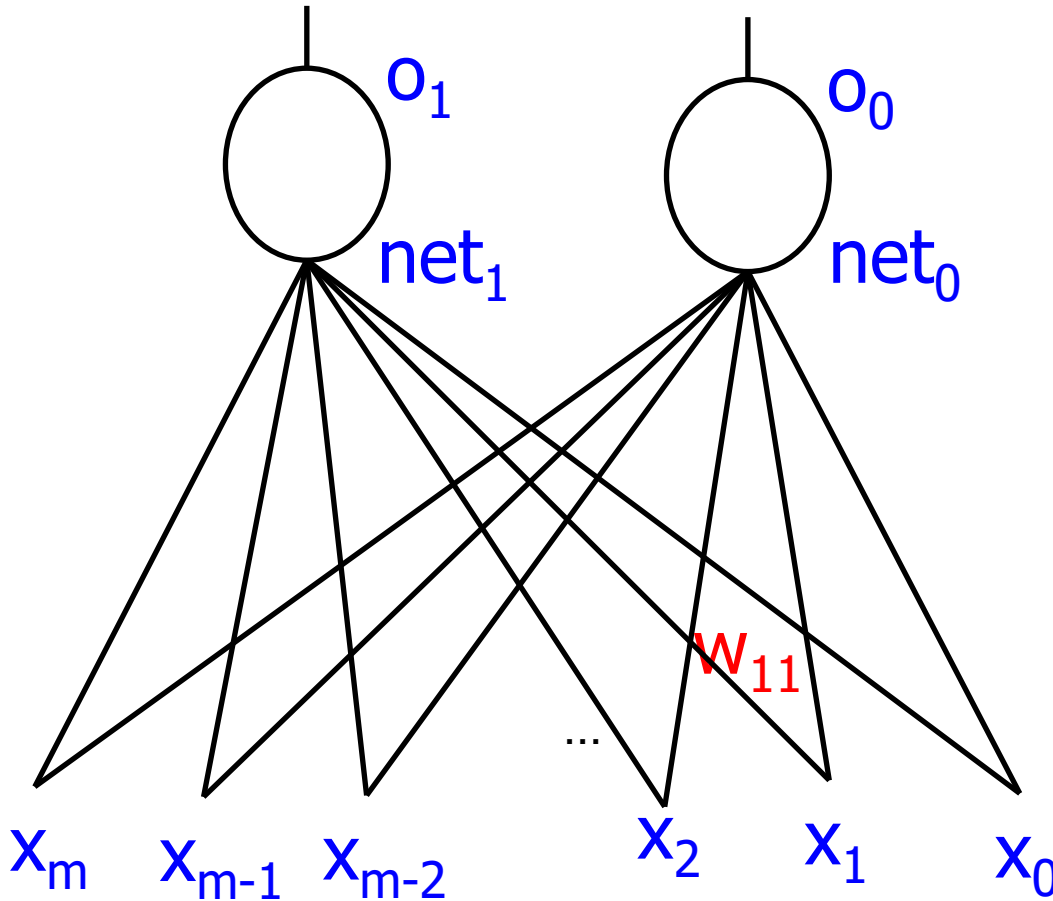
$$o = \frac{1}{1 + e^{-net}} \Rightarrow \frac{\partial o}{\partial net} = o(1-o)$$

$$net = \sum_{j=0}^m w_j x_j \Rightarrow \frac{\partial net}{\partial w_1} = x_1$$

$$\Rightarrow \Delta w_1 = \eta \frac{\partial L}{\partial w_1} = \eta(t-o)x_1$$

$$\Delta w_1 = \eta(t-o)x_1$$

Multiple neurons in the output layer: softmax+*cross entropy* loss (1/2): illustrated with 2 neurons and single training data point



$$O = \langle o_1, o_0 \rangle$$

$$NET = \langle net_1, net_0 \rangle$$

$$o_1 = \frac{e^{net_1}}{e^{net_1} + e^{net_0}}, \quad o_0 = \frac{e^{net_0}}{e^{net_1} + e^{net_0}}$$

$$\frac{\partial O}{\partial NET} = \begin{bmatrix} \frac{\partial o_0}{\partial net_0} & \frac{\partial o_1}{\partial net_0} \\ \frac{\partial o_0}{\partial net_1} & \frac{\partial o_1}{\partial net_1} \end{bmatrix}$$

$$= \begin{bmatrix} o_0(1-o_0) & -o_0o_1 \\ -o_1o_0 & o_1(1-o_1) \end{bmatrix}$$

Softmax and Cross Entropy (2/2)

$$L = -t_1 \log o_1 - t_0 \log o_0$$

$$o_1 = \frac{e^{net_1}}{e^{net_1} + e^{net_0}}, o_0 = \frac{e^{net_0}}{e^{net_1} + e^{net_0}}$$

$$\frac{\partial L}{\partial w_{11}} = -\frac{t_1}{o_1} \frac{\partial o_1}{\partial w_{11}} - -\frac{t_0}{o_0} \frac{\partial o_0}{\partial w_{11}}$$

$$\frac{\partial o_1}{\partial w_{11}} = \frac{\partial o_1}{\partial net_1} \cdot \frac{\partial net_1}{\partial w_{11}} + \frac{\partial o_1}{\partial net_0} \cdot \frac{\partial net_0}{\partial w_{11}} = o_1(1-o_1)x_1 + 0$$

$$\frac{\partial o_0}{\partial w_{11}} = \frac{\partial o_0}{\partial net_1} \cdot \frac{\partial net_1}{\partial w_{11}} + \frac{\partial o_0}{\partial net_0} \cdot \frac{\partial net_0}{\partial w_{11}} = -o_1 o_0 x_1 + 0$$

$$\Rightarrow \frac{\partial L}{\partial w_{11}} = -t_1(1-o_1)x_1 + t_0 o_1 x_1 = -t_1(1-o_1)x_1 + (1-t_1)o_1 x_1$$

$$= [-t_1 + t_1 o_1 + o_1 - t_1 o_1]x_1 = -(t_1 - o_1)x_1$$

$$\Delta w_{11} = -\eta \frac{\partial E}{\partial w_{11}} = \eta(t_1 - o_1)x_1$$

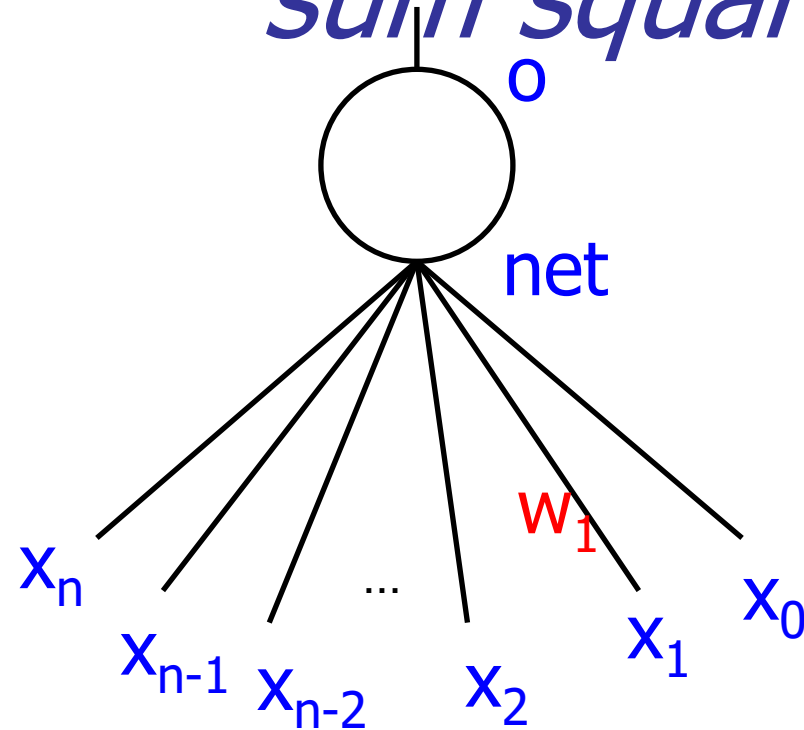
Can be generalized

- When L is Cross Entropy Loss, the change in any weight is

***learning rate *
diff between target and observed
outputs *
input at the connection***

Weight change rule with TSS

Single neuron: *sigmoid+total sum square (tss) loss*



Lets consider wlg w_1 . Change is weight $\Delta w_1 = -\eta \delta L / \delta w_1$
 η = learning rate,

$$L = \text{loss} = \frac{1}{2}(t - o)^2,$$

t = target, o = observed output

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial \text{net}} \cdot \frac{\partial \text{net}}{\partial w_1}$$

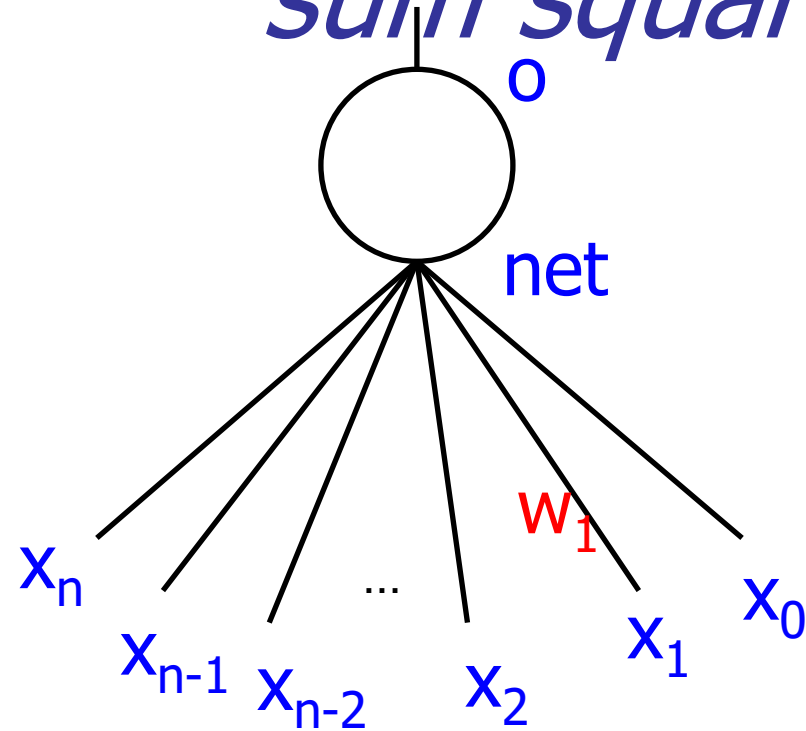
$$L = \frac{1}{2}(t - o)^2 \Rightarrow \frac{\partial L}{\partial o} = -(t - o)$$

$$o = \frac{1}{1 + e^{-\text{net}}} (\text{sigmoid}) \Rightarrow \frac{\partial o}{\partial \text{net}} = o(1 - o)$$

$$\text{net} = \sum_{i=0}^n w_i x_i \Rightarrow \frac{\partial \text{net}}{\partial w_1} = x_1$$

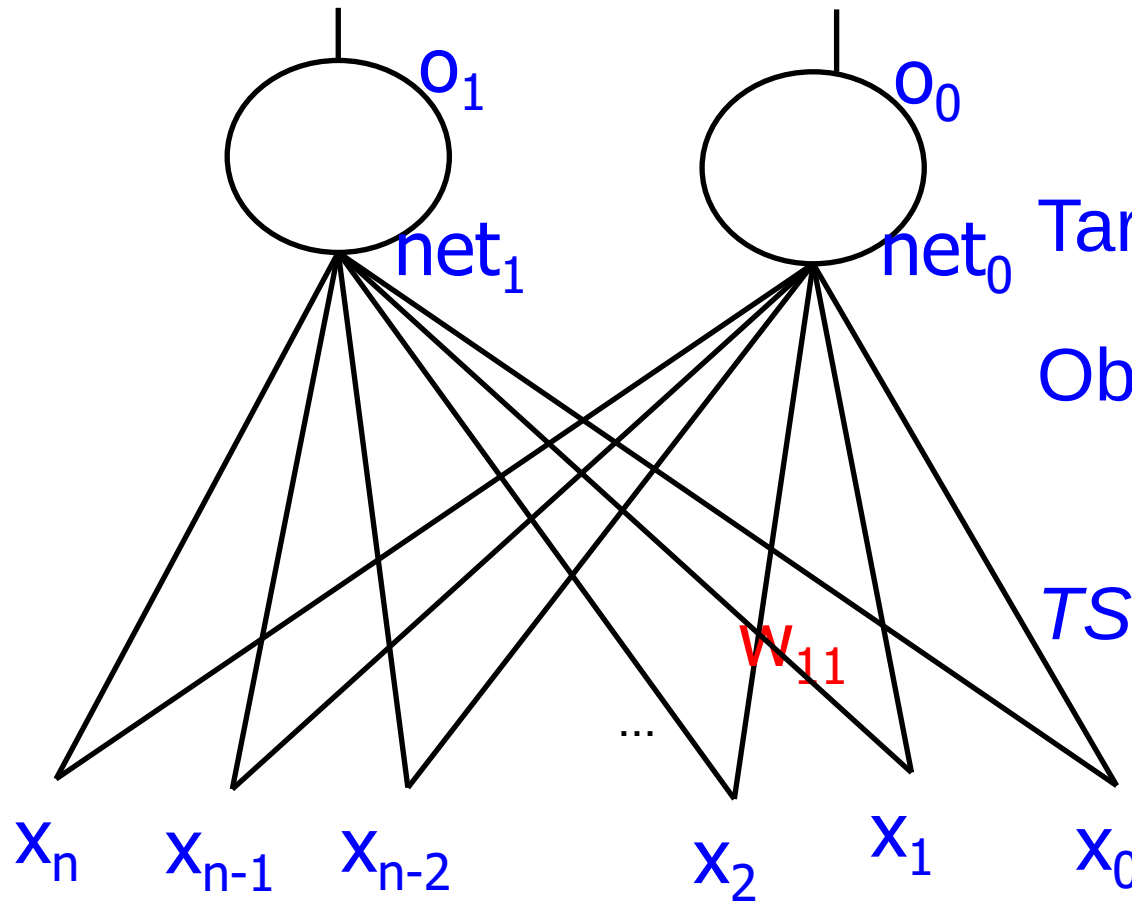
$$\Rightarrow \Delta w_1 = \eta(t - o)o(1 - o)x_1$$

Single neuron: *sigmoid+total sum square* (tss) loss (cntd)



$$\Delta w_1 = \eta(t-o)o(1-o)x_1$$

layer . sigmoid+total sum square (tss) loss



Target vector: $\langle t_1, t_0 \rangle$

Observed vector:
 $\langle o_1, o_0 \rangle$

TSS Loss, $L = \frac{1}{2}[(t_1 - o_1)^2 + (t_0 - o_0)^2]$

$$\Delta w_{11} = \eta(t_1 - o_1)o_1(1 - o_1)x_1$$